

CM 3 — Tableaux et complexité

Algorithmique 1 · L1 Informatique, UPS · Semestre 1 2026

Partie 1 — Le type abstrait Tableau et Matrice

Un **type abstrait** décrit ce qu'on peut faire avec une donnée (ses opérations autorisées), sans dire comment c'est fait en mémoire. C'est ce qui sépare le **client** (le code qui utilise le type) du **prestataire** (le code qui l'implémente) : le client n'a besoin de connaître que les opérations, jamais la représentation interne.

Pourquoi ne pas utiliser directement les `List` de Python ? Parce que le but de l'UE est d'apprendre des notions d'algorithmique **indépendantes du langage**. Si on utilisait toutes les possibilités des listes Python (`append`, `insert`, `pop`...), le raisonnement ne se transposerait plus en C, Java, etc. On se restreint donc volontairement à un socle minimal d'opérations, disponible dans (presque) tous les langages.

Exemple concret : le type `int` en Python a pour espace de valeurs l'ensemble des entiers relatifs ($\mathbb{Z}$) et pour opérations autorisées `+`, `-`, `%`, etc. Le type `bool` a pour espace de valeurs `{ False, True }` et pour opérations `not`, `and`, `or`, `==`. Chaque opération se spécifie comme une fonction : ses paramètres sont les opérandes, sa valeur de retour est le résultat, et sa **pré-condition** / **post-condition** précisent dans quel cas elle s'applique et ce qu'elle garantit.

Vocabulaire : on appelle **type abstrait** la spécification d'un type (ses opérations autorisées), indépendamment de son implémentation. Une **structure de données** définit en plus son implémentation, c'est-à-dire sa mise en oeuvre matérielle (comment elle est représentée en mémoire).

Le type abstrait Tableau

Un tableau mémorise une séquence d'éléments avec 3 règles : le **nombre d'éléments est fixe**, tous les éléments ont le **même type**, et chaque élément est repéré par sa **position (indice)**.

Opération	Ce qu'elle fait	En Python (restreint)
Créer	fixe la longueur et la valeur par défaut	<code>tableau = creer_tableau(longueur, default)</code>
Longueur	donne le nombre d'éléments	<code>len(tableau)</code>
Lire	lit l'élément à l'indice i ($0 \leq i < \text{len}$)	<code>valeur = tableau[i]</code>
Écrire	remplace l'élément à l'indice i	<code>tableau[i] = valeur</code>

C'est tout : **pas d'append, insert ou pop**. Ces méthodes changent la longueur de la liste, ce qui casserait la règle « taille fixe » du tableau.

```
def creer_tableau(longueur, default):
    assert longueur >= 0, 'Pre-condition'
    return [default] * longueur

tableau = creer_tableau(20, 0)    # tableau de 20 entiers, tous à 0
tableau[0] = int(input())        # écriture
tableau[1] = 2 * tableau[0]      # lecture puis écriture
```

Pourquoi ces 4 opérations et pas d'autres ?

Elles collent exactement à ce qu'une **zone contiguë de mémoire** permet de faire efficacement : chaque case occupe une taille fixe, et l'adresse de l'élément i se calcule directement par `adresse(tableau[0]) + i × taille_case`. C'est pour ça qu'en C, un tableau a une taille fixée une fois pour toutes à sa création :

```
int tableau[20] = {0};           // 20 cases, valable uniquement pour 0
scanf("%d", &tableau[0]);        // écriture
tableau[1] = 2 * tableau[0];     // lecture puis écriture
```

Exercice corrigé — la fonction déplacer

```
def deplacer(tableau, i_source, i_destination):
    valeur = tableau.pop(i_source)
    tableau.insert(i_destination, valeur)
```

1) Ce qu'elle fait : elle retire l'élément situé à l'indice `i_source` puis le réinsère à l'indice `i_destination` — elle déplace un élément dans la liste, en décalant tous les éléments entre les deux positions.

2) Pourquoi ça viole le type Tableau : `pop` et `insert` font partie des opérations interdites, car elles changent la longueur de la liste (même si c'est temporaire, le temps d'un appel). Un tableau n'autorise que lire/écrire à un indice existant.

3) Correction : on ne peut utiliser que lire et écrire. On recopie donc « à la main » les éléments un par un, en décalant dans le bon sens :

```
def deplacer(tableau, i_source, i_destination):
    assert 0 <= i_source < len(tableau), 'Pre-condition'
    assert 0 <= i_destination < len(tableau), 'Pre-condition'
    valeur = tableau[i_source]
    if i_source < i_destination:
        for i in range(i_source, i_destination):
            tableau[i] = tableau[i + 1]
    else:
        for i in range(i_source, i_destination, -1):
            tableau[i] = tableau[i - 1]
    tableau[i_destination] = valeur
```

4) Contrainte à garantir : tous les éléments doivent être conservés, seulement réordonnés — ni perdu, ni dupliqué. La version corrigée le garantit naturellement : elle ne fait que des décalages internes au tableau, dont la longueur ne change jamais.

Le type abstrait Matrice

Une matrice, c'est un tableau à **deux dimensions** (lignes × colonnes), utile pour représenter des données 2D (images, grilles, transformations géométriques...). En Python restreint, on la représente comme une liste non vide de listes non vides, toutes de même longueur :

```
def creer_matrice(nb_lignes, nb_colonnes, default):
    assert nb_lignes > 0 and nb_colonnes > 0, 'Pre-condition'
    matrice = [None] * nb_lignes
    for i in range(nb_lignes):
        matrice[i] = [default] * nb_colonnes
    return matrice

matrice = creer_matrice(19, 13, 0)    # 19 lignes, 13 colonnes
nb_lignes = len(matrice)
nb_colonnes = len(matrice[0])
matrice[3][4] = int(input())          # écriture à la ligne 3, colonne 4
```

Même logique que pour le tableau : dimensions et type fixés à la création, donc aucune méthode qui changerait la longueur d'une des listes n'est autorisée.

Deux motivations concrètes pour ce type : représenter des données en 2D (une image en niveaux de gris est une matrice de pixels, un écran une matrice de couleurs), et représenter la notion mathématique de matrice, utilisée par exemple pour les transformations du plan ou de l'espace. Une rotation 3D peut ainsi s'écrire comme une matrice 3×3 :

```
rot = [
    [ 0, 1, 0],
    [-1, 0, 0],
    [ 0, 0, 1]
]
```

Partie 2 — Complexité

La **complexité** d'un algorithme, c'est la quantité de ressources (temps, mémoire) qu'il consomme, exprimée **en fonction de la taille de son entrée** (longueur d'un tableau, valeur d'un entier...). Plusieurs algorithmes peuvent

résoudre le même problème : la complexité permet de choisir le plus adapté, en particulier pour de grandes entrées.

Rappels de vocabulaire

- **Problème** : caractérisé par sa spécification, c'est-à-dire une pré-condition (ce qu'on suppose sur l'entrée) et une post-condition (ce que doit garantir la sortie).
- **Implémentation** : une fonction concrète qui respecte cette spécification.
- **Algorithme** : l'ensemble des opérations qui permettent de respecter la spécification, indépendamment du langage de programmation utilisé.

Mesurer expérimentalement : l'exemple somme_entiers

```
def somme_entiers_1(n):  
    somme = 0  
    for i in range(1, n + 1):  
        somme = somme + i  
    return somme
```

n	Temps mesuré	Facteur vs ligne précédente
10 ⁵	0,005 s	—
10 ⁶	0,047 s	× 9,4
10 ⁷	0,428 s	× 9,1
10 ⁸	4,37 s	× 10,2
10 ⁹	42,5 s	× 9,7
10 ¹⁰	457 s	× 10,7

Interprétation (question 4) : à chaque fois que n est multiplié par 10, le temps l'est aussi (facteur proche de 10 sur toute la table). C'est la signature d'une complexité **linéaire**, notée $O(n)$: la fonction fait bien une boucle de n itérations, donc son temps est proportionnel à n .

Estimation pour $n = 10^{50}$ (question 5) : le temps est proportionnel à n . Comme 10^{10} prend 457 s, et que 10^{50} est 10^{40} fois plus grand, il faudrait environ 457×10^{40} secondes, soit de l'ordre de 10^{35} années — bien plus que l'âge de l'univers. C'est un algorithme en $O(n)$ tout à fait correct, mais totalement impraticable pour de très grandes entrées.

D'où l'intérêt de `somme_entiers_2`, qui utilise la formule $n(n+1)/2$: un seul calcul, quelle que soit la taille de n . Sa complexité est **constante**, $O(1)$: sur la même machine, l'appel avec $n = 10^{50}$ prend environ $1,9 \times 10^{-6}$ s, comme pour n'importe quel n .

Application : la racine carrée entière

Problème : étant donné un entier positif n , déterminer la partie entière de sa racine carrée.

Spécification

- Type du paramètre : n est un entier · Type du retour : a est un entier
- Pré-condition : $n \geq 0$
- Post-condition : $a \geq 0$ and $a^2 \leq n < (a+1)^2$

Algorithme 1 — par balayage : on part de $a = 0$ et on incrémente jusqu'à trouver la solution.

```
def racine_entiere_1(n):  
    assert n >= 0, 'Pre-condition'  
    a = 0  
    while (a + 1)**2 <= n:  
        a += 1  
    assert a >= 0 and a**2 <= n < (a + 1)**2, 'Post-condition'  
    return a
```

Algorithme 2 — par dichotomie : on établit une zone de recherche $[a, b]$ contenant la solution. On calcule le carré du milieu $m = (a+b)/2$: s'il est inférieur à n , la solution est dans la moitié droite, sinon dans la moitié gauche. On répète en coupant l'intervalle en deux jusqu'à ce qu'il soit de longueur 1. À l'initialisation, $a = 0$ et $b =$

n+1.

Algorithme	Nombre d'itérations	Complexité
Balayage	de l'ordre de $\sqrt{n}$	$O(\sqrt{n})$
Dichotomie	de l'ordre de $\log_2(n)$	$O(\log_2 n)$

Exemple de déroulé de la dichotomie (n = 27) :

a	b	m	m ²	m ² ≤ n ?
0	28	14	196	Faux
0	14	7	49	Faux
0	7	3	9	Vrai
3	7	5	25	Vrai
5	7	6	36	Faux
5	6	—	—	zone de longueur 1 : a = 5

Exercice — écrire racine_entiere_2 par dichotomie

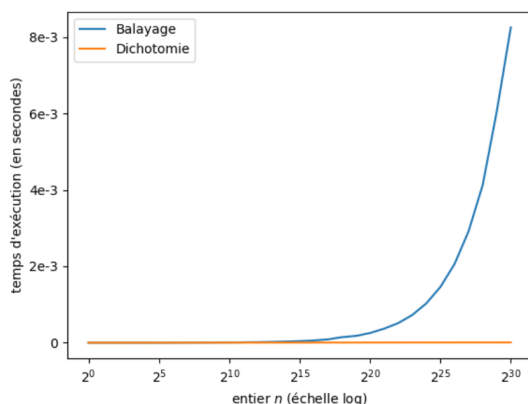
Même spécification que racine_entiere_1, mais avec l'algorithme par dichotomie. Cas particulier utile à vérifier : n = 1. On initialise a = 0, b = n+1 = 2.

```
def racine_entiere_2(n):
    assert n >= 0, 'Pre-condition'
    a, b = 0, n + 1
    while b - a > 1:
        m = (a + b) // 2
        if m**2 <= n:
            a = m
        else:
            b = m
    assert a >= 0 and a**2 <= n < (a + 1)**2, 'Post-condition'
    return a
```

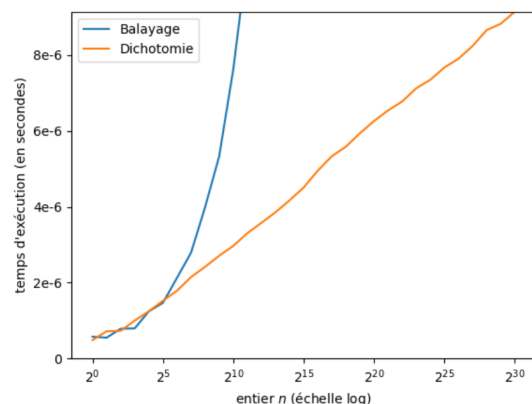
Déroulé pour n = 1 : a=0, b=2 → m=1, m²=1 ≤ 1 donc a=1 → b-a=1, on s'arrête : a=1. On vérifie bien 1² ≤ 1 < 2². Les cas n = 0 et n = 17 se déroulent sur le même principe (n = 0 s'arrête immédiatement, a = 0).

Nombre d'itérations dans le pire des cas : à chaque tour, la longueur de la zone de recherche est divisée par 2 ; elle part de n+1 et s'arrête à 1. Il faut donc environ $\log_2(n)$ itérations, contre $\sqrt{n}$ pour le balayage — nettement moins dès que n est grand.

Les deux graphes du cours (temps d'exécution en fonction de n, échelle logarithmique en x) confirment cet écart :



Sur toute la plage (n jusqu'à 2³⁰) : le balayage explose, la dichotomie reste quasi plate.



Zoom sur les petits n : pour de très petites entrées, les deux algorithmes se valent.

Complexités usuelles à connaître

Situation	Exemple concret	Complexité
-----------	-----------------	------------

Parcours linéaire d'un tableau de longueur n	chercher le maximum d'une liste	$O(n)$
Boucles imbriquées sur deux indices de 1 à n	comparer chaque élément à chaque autre	$O(n^2)$
Parcours complet d'une matrice $n_l \times n_c$	additionner tous les éléments d'une image	$O(n_l \times n_c)$
Recherche dichotomique dans un tableau trié	chercher un mot dans un dictionnaire trié	$O(\log_2 n)$
Tris quadratiques (insertion, sélection)	trier une petite liste « à la main »	$O(n^2)$
Tris diviser-pour-régner (fusion, rapide)	trier une grande liste efficacement	$O(n \times \log_2 n)$

À retenir

- **Un tableau/une matrice, c'est 4 opérations et rien d'autre** : créer, (dimensions), lire, écrire. Toute méthode qui changerait la taille (append, insert, pop) est hors-jeu.
- **Pour reconnaître une complexité expérimentalement** : multiplie n par 10 et regarde le facteur sur le temps — facteur $\approx 10 \rightarrow O(n)$, facteur $\approx 100 \rightarrow O(n^2)$, facteur quasi nul $\rightarrow O(1)$ ou $O(\log n)$.
- **Balayage vs dichotomie** : même résultat, complexités très différentes ($O(\sqrt{n})$ contre $O(\log_2 n)$). Un bon algorithme peut rendre praticable ce qu'un algorithme correct mais naïf ne permet pas.
- **Retenir le tableau des complexités usuelles** : il revient tout le temps pour estimer la complexité d'un nouveau code, sans avoir à le mesurer.